Lab Procedure
IMU

Introduction

Ensure the following:

1. You have reviewed the Application Guide – IMU
2. Make sure the **Mechatronic Sensors Trainer** is out of its case; it is powered using the provided USB cable connected to the PC.
    a. The LED next to the USB C port will stay white after the touch screen starts loading.
    b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.

Analyzing Raw IMU Data & Calibrating

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **`File > Open Folder`**..., and browse to the directory containing the python files and lab procedure for the IMU lab (…/sensors_trainer/1_fundamentals /imu/hardware/python). Open this directory.
2. Open imu_scope.py. Ensure that the **`Sensors Trainer`** is sitting vertically straight on your desk. Ensure that the **`totalTime`** parameter on the `timer = Timer(sampleRate=480, totalTime=20)` line is set to 20 seconds. These values are set on the experiment constants section of the script. Run this script using the  button or the terminal but leave the **`Sensors Trainer`** untouched. This will open 2 plots **`Accelerometer`** and **`Gyroscope`**. When the script terminates after 20 seconds, it prints out the mean and standard deviation of the IMU data over the 20 seconds. Why is this information important? Repeat this step 3 times and record the means and standard deviations.
3. Adjust the **`bias`** variable based on values you estimated in the previous step for the gyroscope alone, add the values in X, Y and Z. Run the script for 20 seconds and confirm that the gyroscope means are closer to 0.
4. Set the **`totalTime`** parameter in the **`timer`** initialization now to 300 seconds and re-run the script. Look at the **`Accelerometer`** plot. What are the units of the signals you are observing? Are any of the three signals significantly non-zero? Note down your findings.
5. Lay down the unit flat (the LCD should be on facing upwards). Which signal is now significantly non-zero? Why is this? Note down your findings.

6. Rotate the Sensors Trainer as required to have approximately +9.81 m/s^2 displayed in one of the three axes while the values in the other two axis are approximately $0\ m/s^2$. What do these three principal axes correspond to?

7. Stop the script using **Ctrl+C** and clicking enter when prompted (the script also automatically stops after 300 seconds).

8. Modify the `accelerometer = sensors.accelerometer` line to divide the accelerometer data by 9.81. Run the script again and note down the units of the accelerometer data. Note that the scopes will still display m/s^2 although the physical units should now be different, do not pay attention to the printed m/s^2 units. Why is measuring in this new scale useful?

9. Look at the **Gyroscope** plot. Rotate the Sensors Trainer about the 3 principal axes identified in step 5. Do this slowly at first, and then rapidly. Note down the direction of positive and negative readings.

10. Stop the code using **Ctrl+C** (the script also terminates automatically after 300 seconds) and close imu_scope.py.

Visualizing IMU Data

1. In the same directory, open imu_visualization.py. Ensure that near the top of the script, **visualization** is set to 1. Run this script using the ▷ button.

2. Rotate the **Sensors Trainer** about the IMU's X axis. How does the square rotate? Does it stay rotated when you stop moving?

3. Hold down **Button 0** on the **Sensors Trainer** and repeat step 2 about the IMU's Y axis. Do the same with **Button 1** about the IMU's Z axis.

4. Stop the code using **Ctrl+C** (the script also terminates automatically after 300 seconds).

5. Set the **visualization** variable to 2. Run the script. Abruptly move the Sensors Trainer in the X, Y and Z directions. What does the visualization demonstrate? Stop the script when done using **Ctrl+C**.

6. Set the **visualization** variable to 3. Run the script. Rotate the **Sensor Trainer** about the X axis, while keeping it vertically straight (ensure that the trainer's Y and Z axis are simultaneously in a consistent vertical plane. What does the visualization demonstrate? Stop the script when done using **Ctrl+C**. Close imu_visualization.py.

Estimating Attitude with IMU

1. In the same directory, open imu_attitude_estimation.py. Note that all 5 elements of the **attitude** variable are set to 0 by default.

2. Modify the **bias** variable values that are 0 to to the values you found when running the imu_scope.py file.

3. Complete **`attitude[0]`** and **`attitude[1]`** using the accelerometer data to estimate the roll & pitch angles. Refer to the 006_inertial_measurement_units concept review for more information. Also ensure that the result is in degrees.

4. Complete **`attitude[2], attitude[3]`** and **`attitude[4]`** using the gyroscope data to estimate the roll, pitch and yaw angles. Refer to the 006_inertial_measurement_units concept review for more information. Also ensure that the result is in degrees. Hint: use the **`sampleRate`** parameter in a discrete integrate, for example,
`attitude = attitude + ( rate x sampleTime )`

5. Run this script using the  button. Starting from a vertical trainer (Z axis should point straight up), rotate the trainer about the X axis alone, and then the Y axis alone. Does the data in the **`Attitude`** plot display accurate rotations for roll and pitch? How do they compare to the corresponding Gyroscopic estimates for roll and pitch?

6. Starting back from a vertical trainer (Z axis should point straight up), rotate the trainer approximately 45 degrees about its X axis (a roll operation), followed by another 45 degrees about its body Y axis (a pitch operation). Does the **`Attitude`** plot show accurate compound rotations for roll and pitch? How do they compare to the corresponding Gyroscopic estimates for roll and pitch?

7. Starting back from a vertical trainer (Z axis should point straight up), rotate the trainer approximately 90 degrees about its X axis (a roll operation). What happens to the pitch data in the **`Attitude`** plot? Why? Does the same behavior occur for gyroscopic pitch?

8. Starting back from a vertical trainer (Z axis should point straight up), rotate the trainer approximately 90 degrees about its Y axis (a pitch operation). What happens to the roll data in the **`Attitude`** plot? Why? Does the same behavior occur for gyroscopic roll?

9. Stop the script when done using **`Ctrl+C`**. Close the imu_attitude_estimation.py script.

10. Power OFF the **`Mechatronic Sensors Trainer`** by disconnecting its USB C cable, disconnect any external sensors and pack them along the base and the USB cables in the provided case.